CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

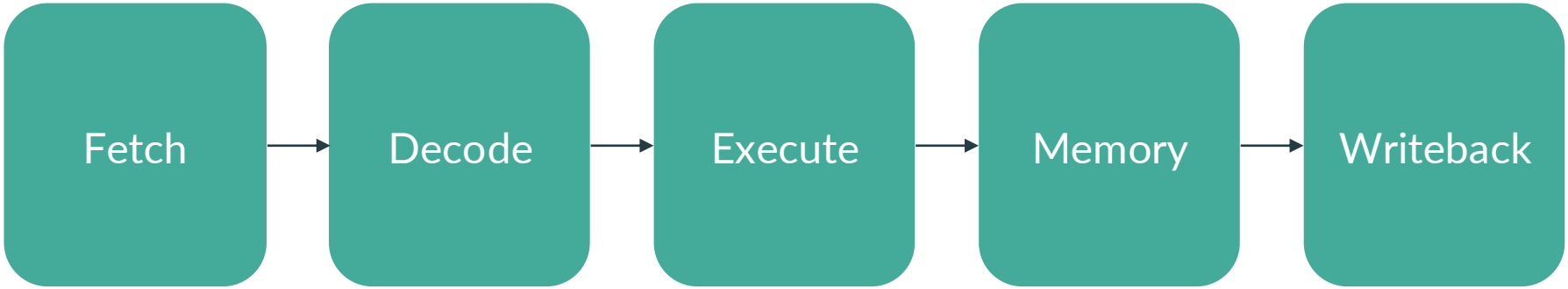
Instruction Execution Cycle

Instruction Execution Cycle

Program Execution

The CPU executes a program by repeatedly executing the next instruction in memory for as long as the program runs.

Each instruction goes through the same stages:



Instruction Execution Cycle

To execute each instruction, the CPU will:

1. Fetch: Retrieve the next instruction from memory.

Instruction Execution Cycle

To execute each instruction, the CPU will:

1. Fetch: Retrieve the next instruction from memory.
2. Decode: Interpret the instruction bits to generate control signals and read operand values from the register file.

Instruction Execution Cycle

To execute each instruction, the CPU will:

1. Fetch: Retrieve the next instruction from memory.
2. Decode: Interpret the instruction bits to generate control signals and read operand values from the register file.
3. Execute: The ALU performs the operation specified by the instruction.

Instruction Execution Cycle

To execute each instruction, the CPU will:

1. Fetch: Retrieve the next instruction from memory.
2. Decode: Interpret the instruction bits to generate control signals and read operand values from the register file.
3. Execute: The ALU performs the operation specified by the instruction.
4. Memory: Access memory to read or write data, if required by the instruction.

Instruction Execution Cycle

To execute each instruction, the CPU will:

1. Fetch: Retrieve the next instruction from memory.
2. Decode: Interpret the instruction bits to generate control signals and read operand values from the register file.
3. Execute: The ALU performs the operation specified by the instruction.
4. Memory: Access memory to read or write data, if required by the instruction.
5. Writeback: Store the result of the executed instruction.

Instruction Cycle Example - Add

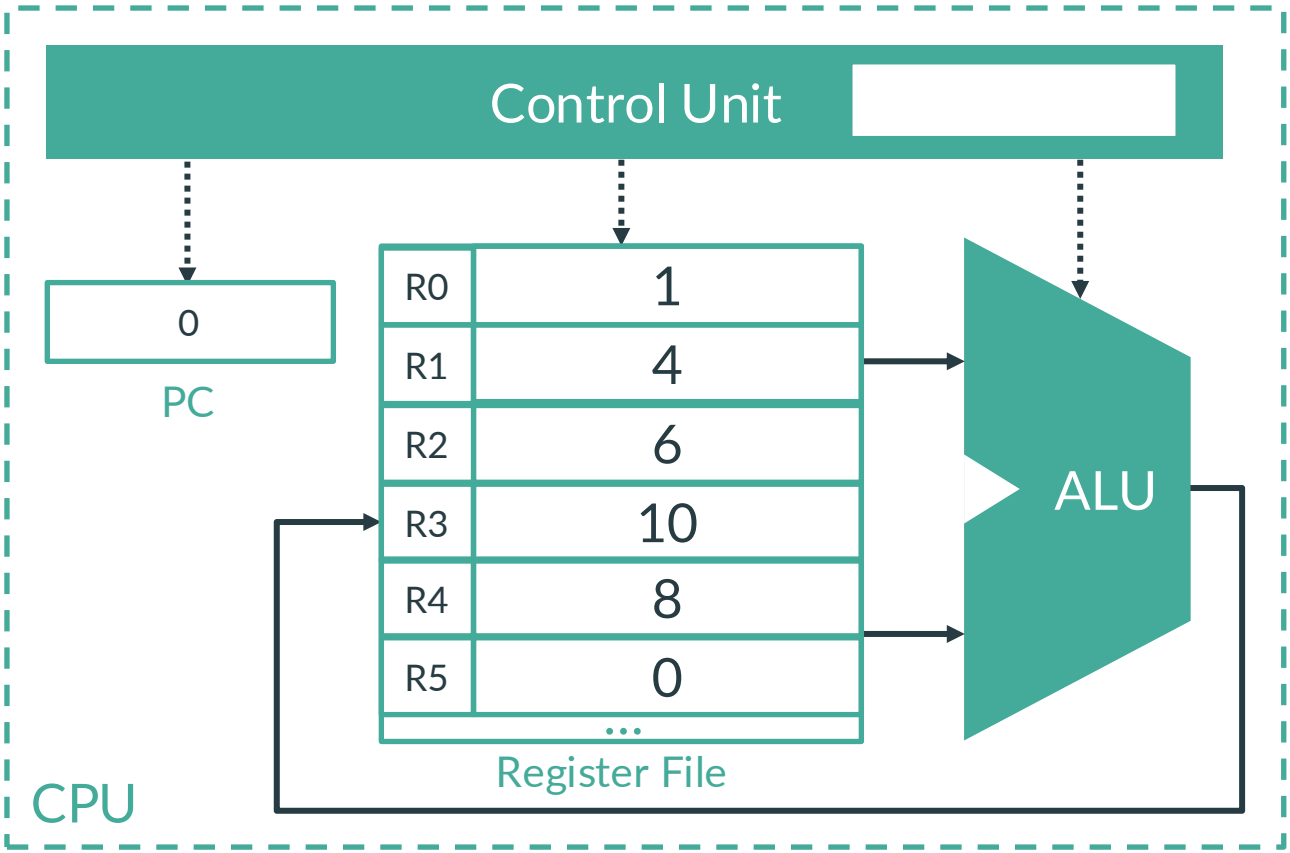
Example: Add Instruction

The instruction `add R3, R0, R1` tells the CPU to add the values in register R0 and register R1 and store the result in register R3.

After executing an add instruction, the CPU will execute the next instruction in memory.

Example: Add Instruction

Initial state:

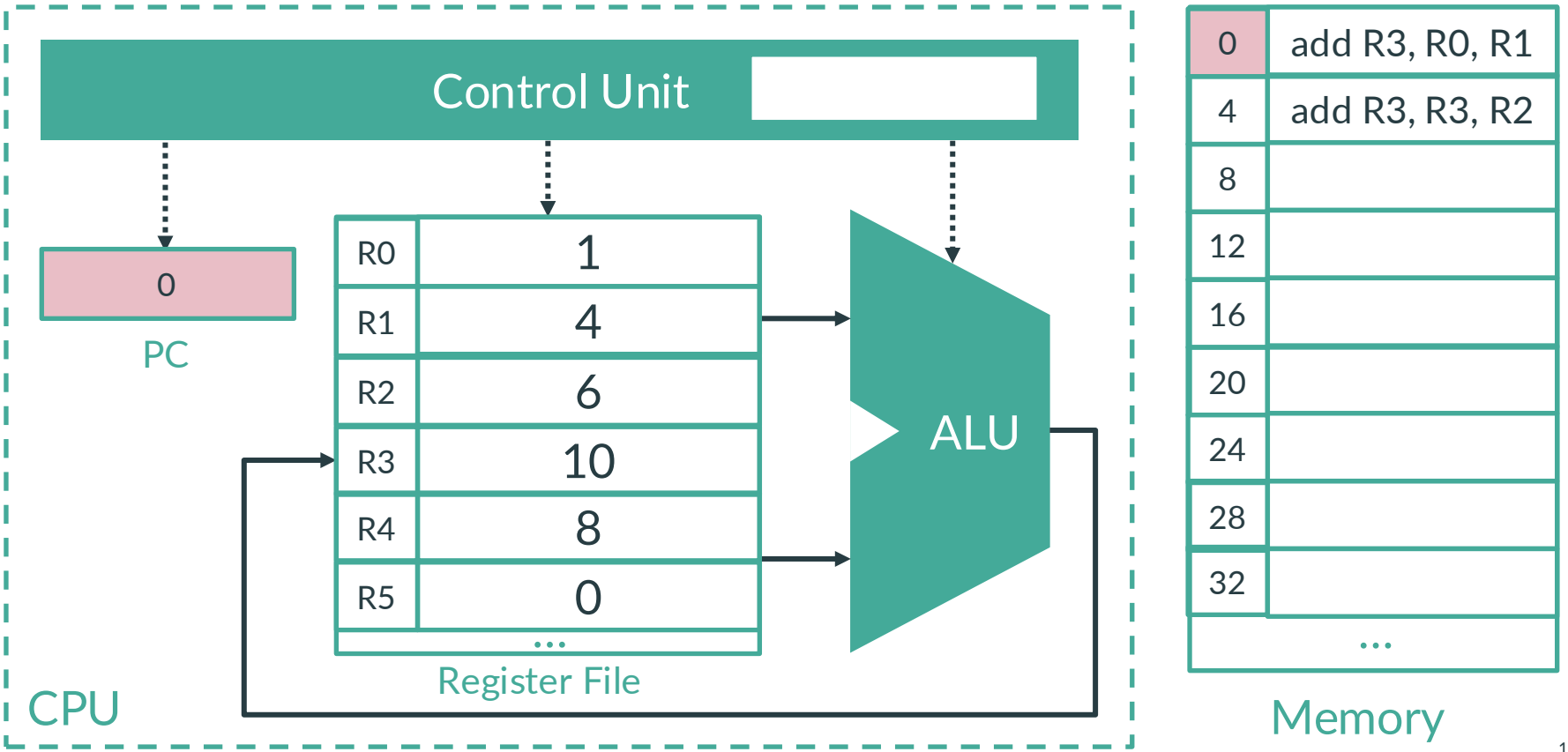


0	add R3, R0, R1
4	add R3, R3, R2
8	
12	
16	
20	
24	
28	
32	
...	

Memory

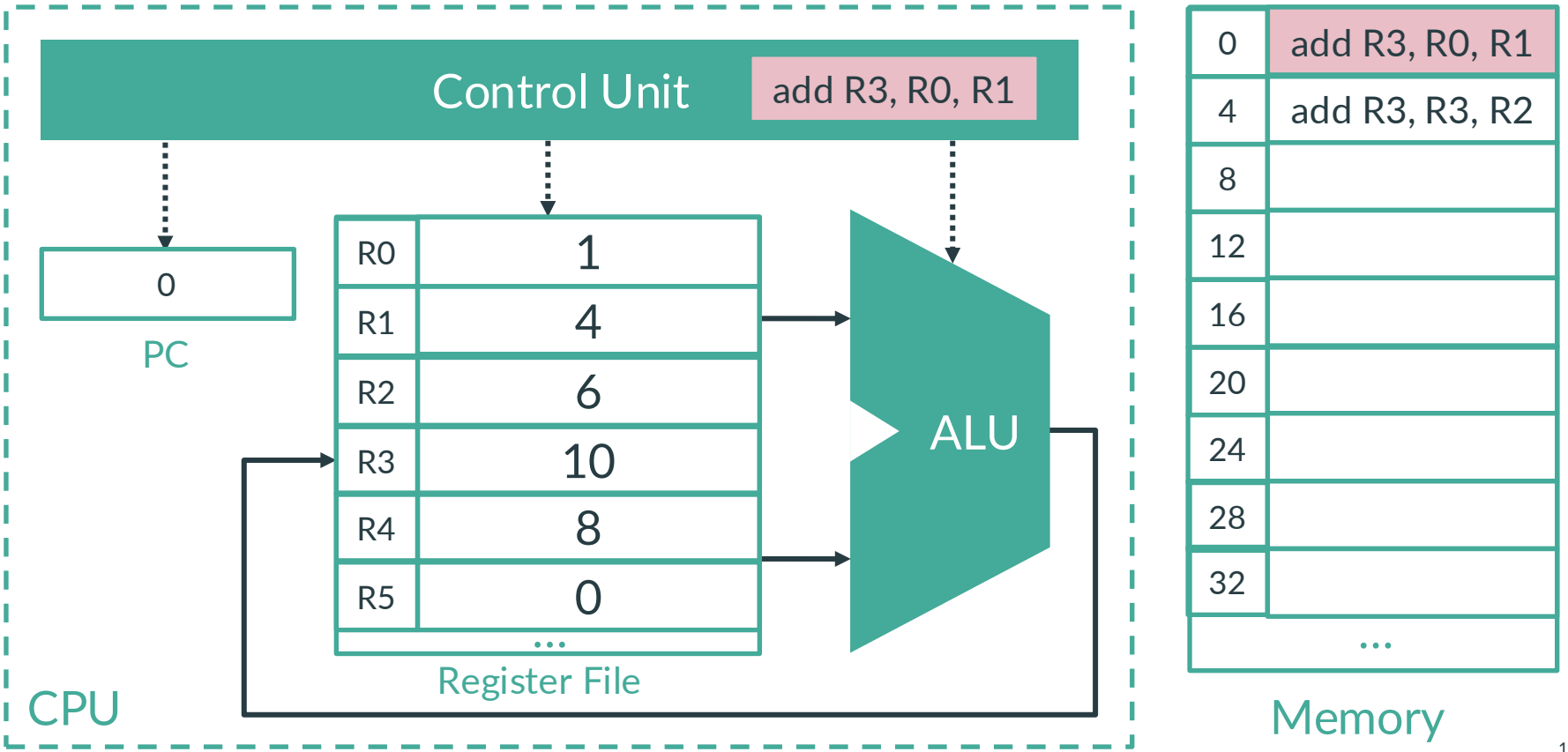
Example: Add Instruction - Fetch

The CPU sets the address bus to the PC value and and the control bus to read from memory



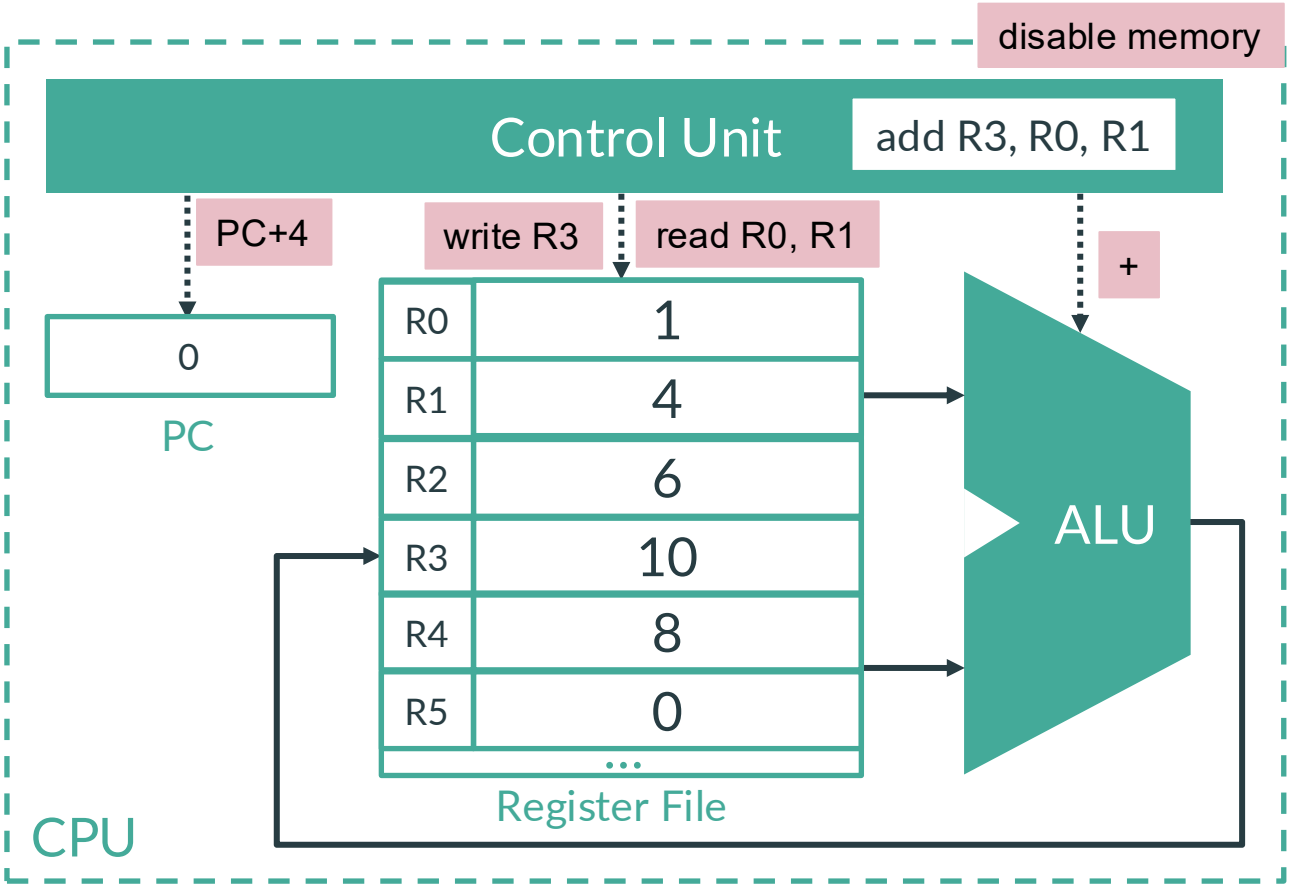
Example: Add Instruction - Fetch

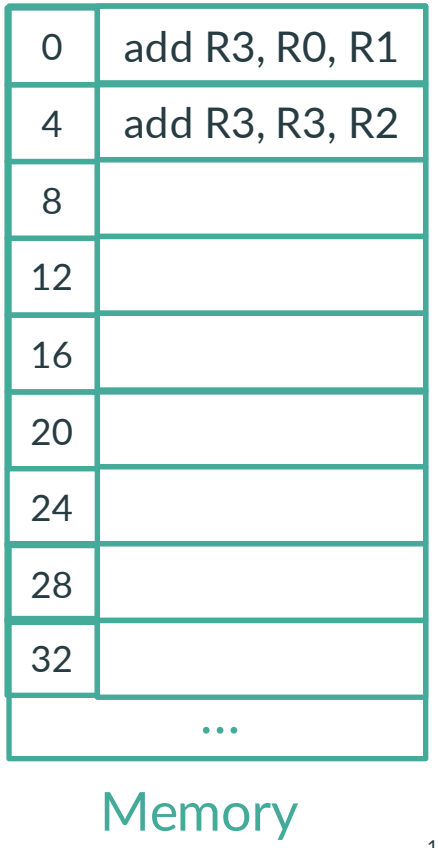
Memory sends the data at memory address 0 to the CPU over the data bus



Example: Add Instruction - Decode

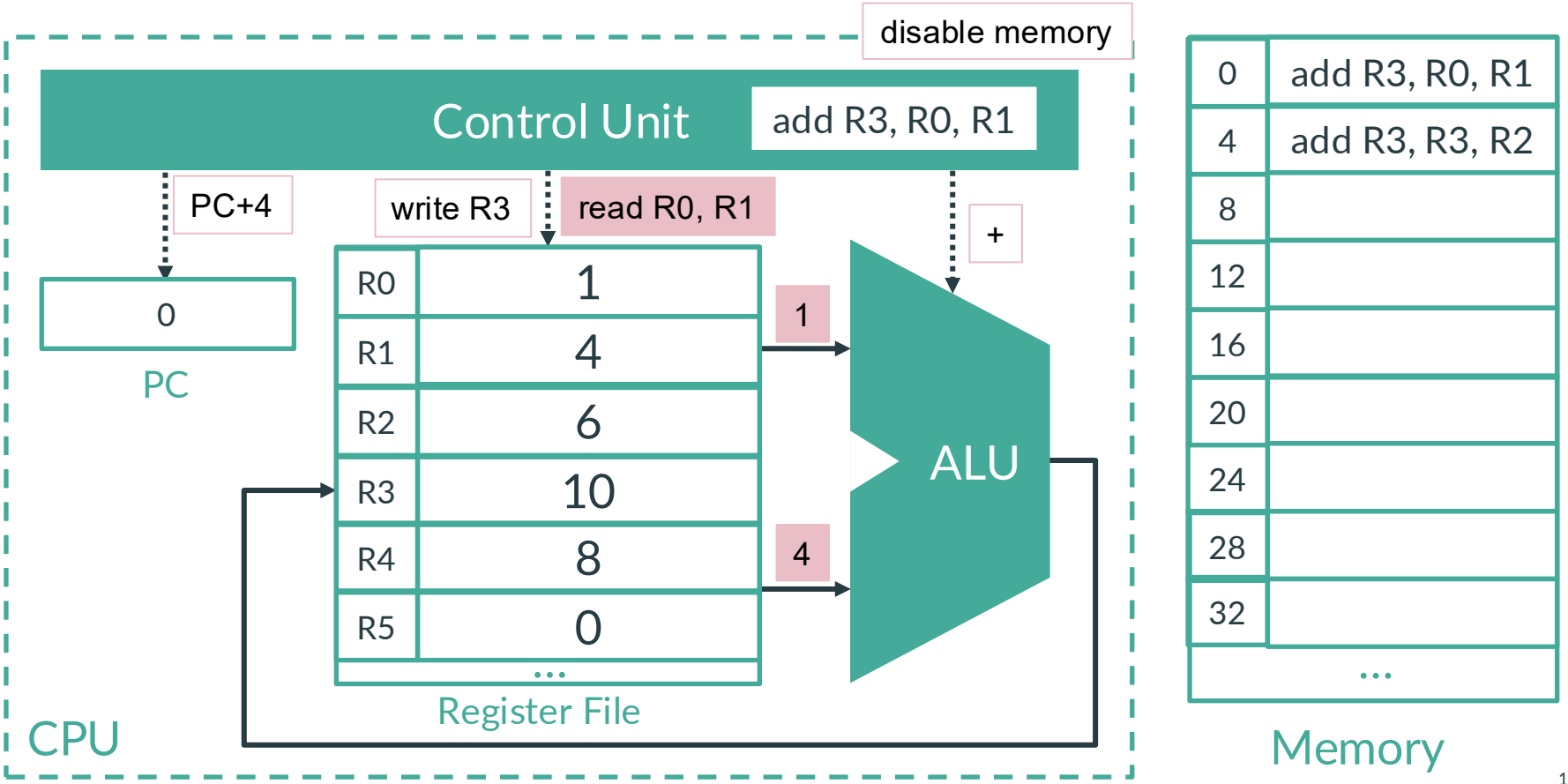
Interpret the instruction bits to generate control signals – next PC value, write register, read register, operation, enable/disable memory access





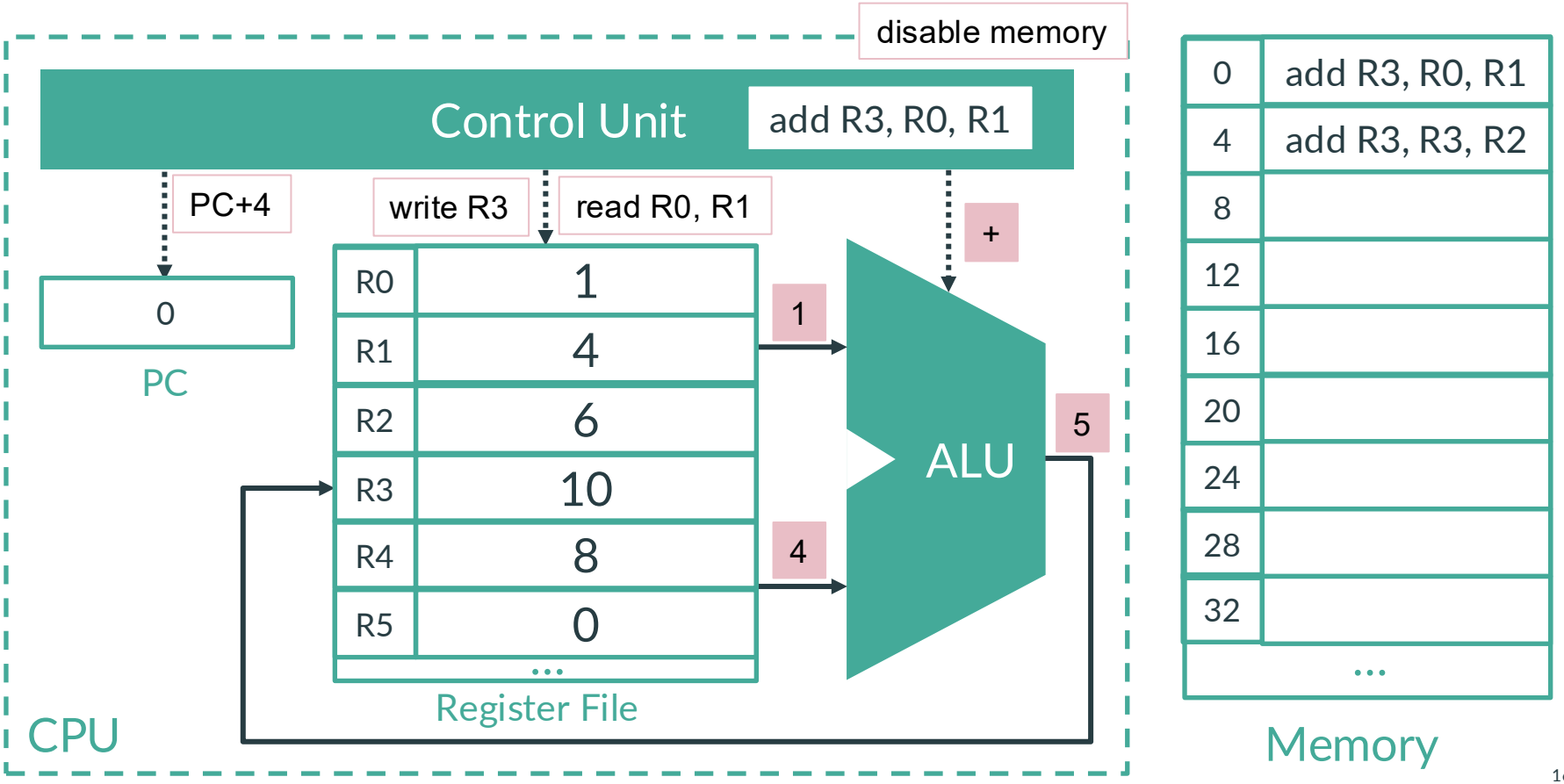
Example: Add Instruction - Decode

Read operand values from register file to prepare for execute stage



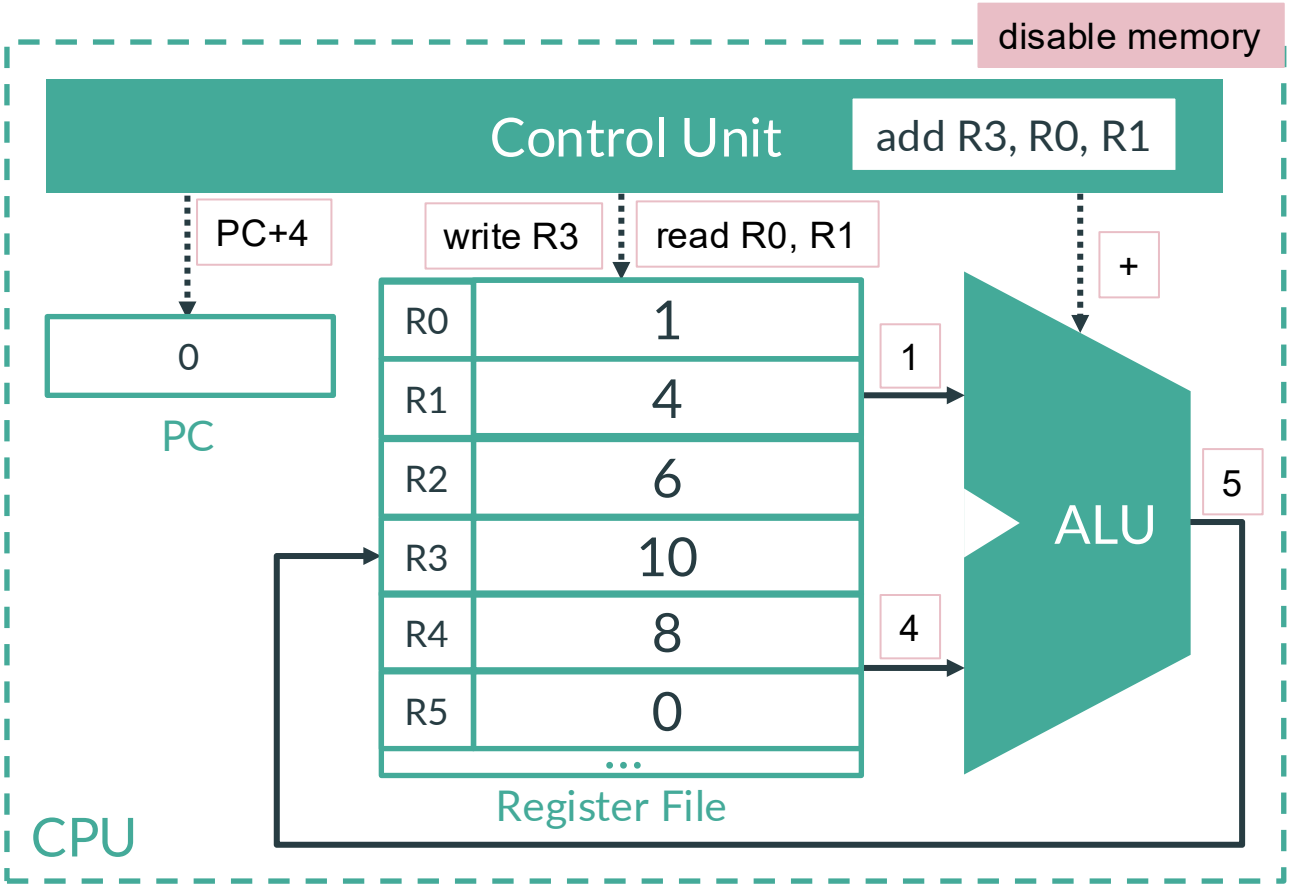
Example: Add Instruction - Execute

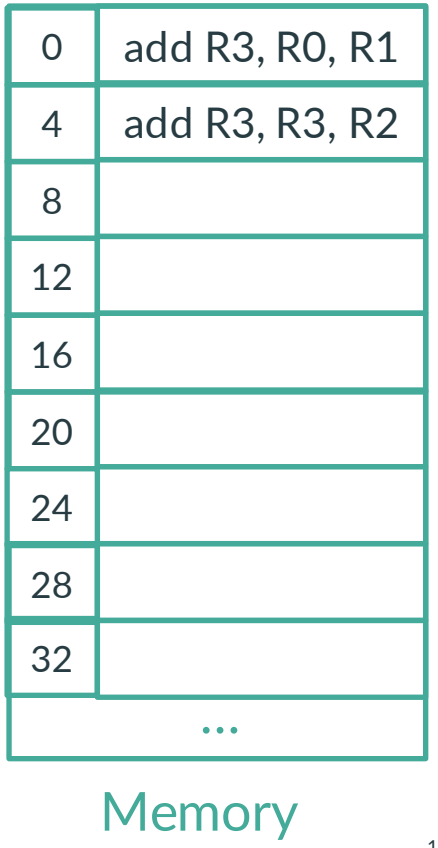
Perform ALU operation



Example: Add Instruction - Memory

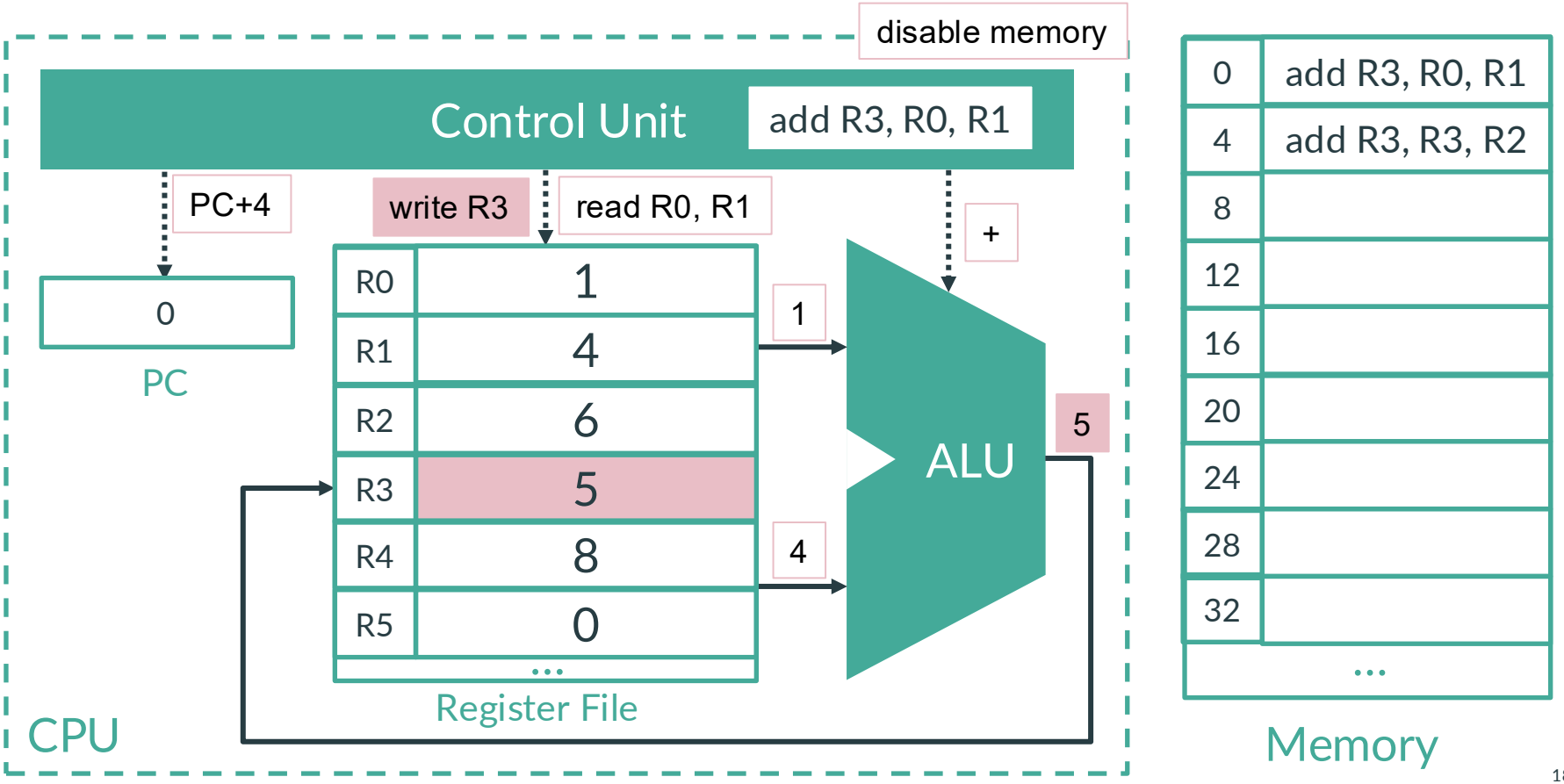
Add instruction does not read or write data to memory, so nothing happens here.





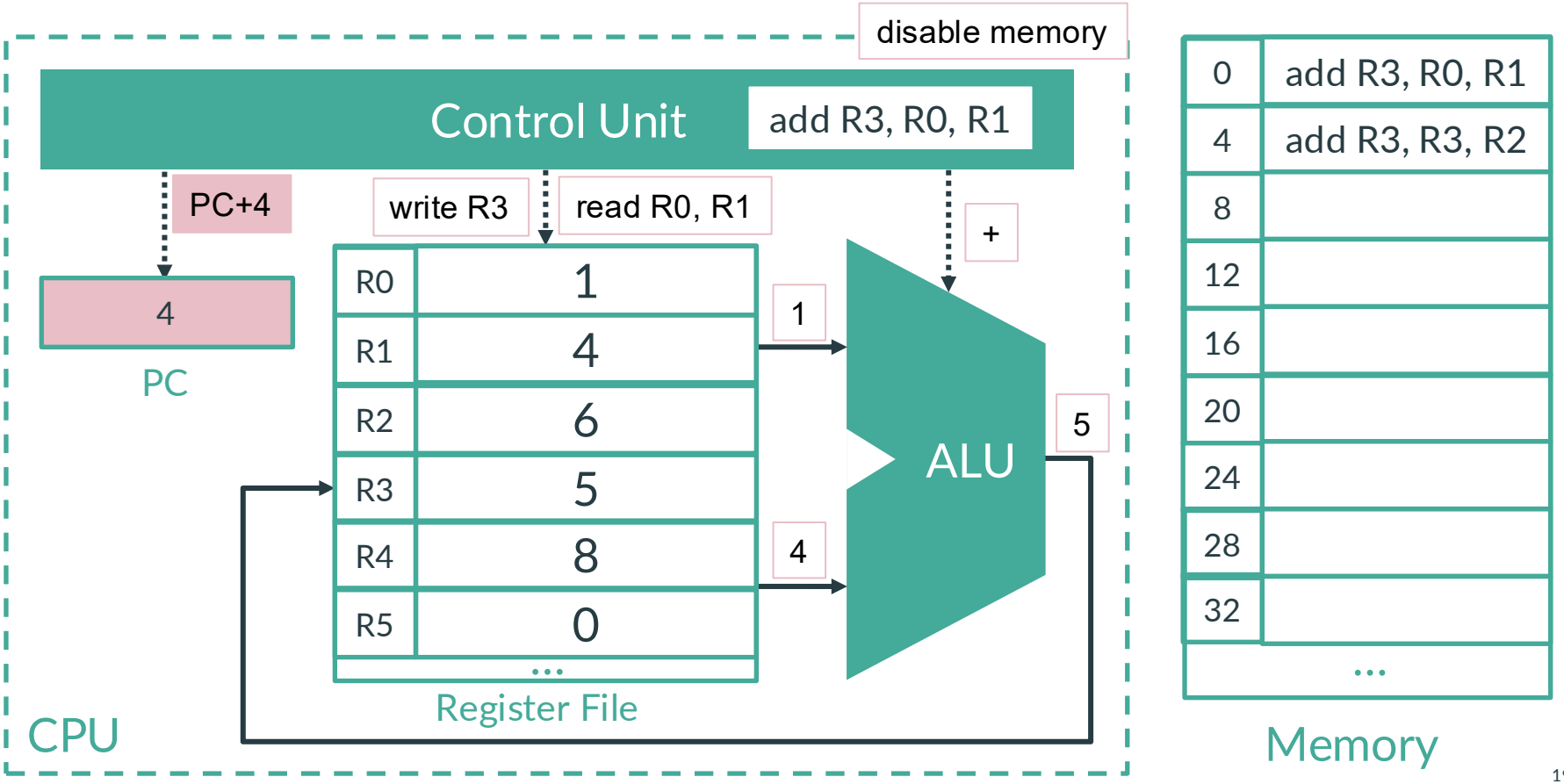
Example: Add Instruction - Writeback

Write result of ALU op to register file



Example: Add Instruction - Writeback

Update PC to hold the address of the next instruction



Exercise: Branch Instruction

CPU Performance

What Determines Performance?

The program execution depends on three things:

- the number of instructions in the program

  fewer instructions ➔ faster program

- the number of clock cycles it takes to execute an instruction

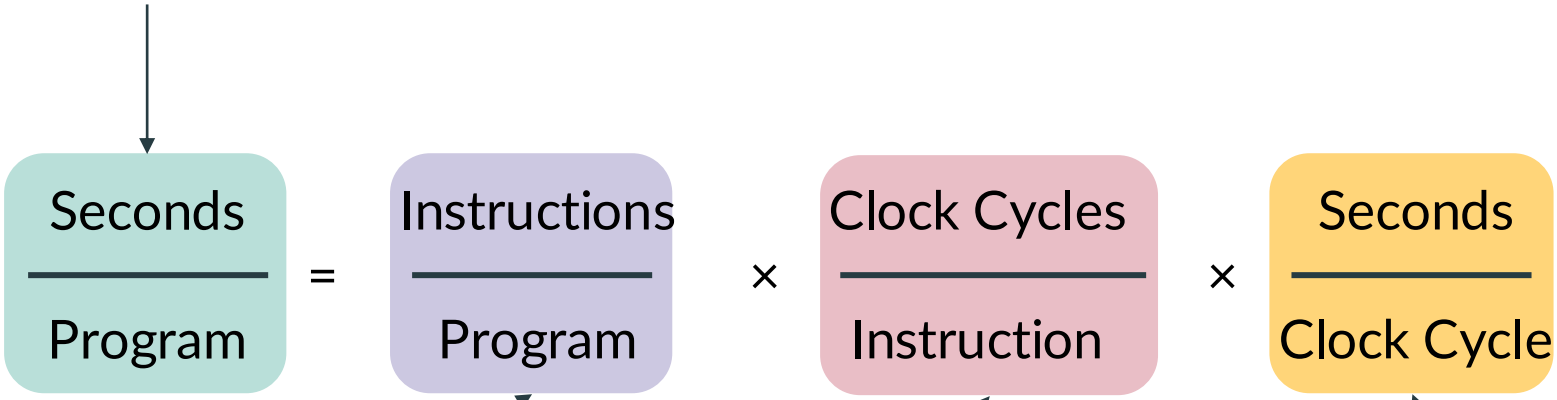
  fewer cycles per instruction ➔ faster execution

- the length of a clock cycle

  higher clock frequency ➔ shorter cycles

Program Execution Time

How long it takes to complete a program



Number of instructions that must execute

Number of clock cycles it takes to execute one instruction

Number of seconds per clock cycle (1/clock frequency)

Not All Instructions Are Equal

Not all instructions take the same amount of time:

- add: entirely in the CPU
- load: memory access can be slow

Clock cycles per instruction (CPI) is the average number of clock cycles per instruction for a program.

Speedup

Speedup is a measure of relative performance:

$$\text{Speedup} = \frac{\text{baseline execution time}}{\text{optimized execution time}}$$

Speedup

Speedup is a measure of relative performance:

$$\text{Speedup} = \frac{\text{baseline execution time}}{\text{optimized execution time}}$$

If B is n times faster than A:

$$\text{Speedup} = \frac{\text{execution time of A}}{\text{execution time of B}} = n$$